Using the same raw UTF-16 bytes of `$utf16name` as the previous example, the code above demonstrates that not only does `mb_strstr()` not require any information about the encoding of the input strings, but it also does not need to be called at all! Provided that both string arguments match encoding, a comparison of byte values is all that is required; neither `strstr()` nor `mb_strstr()` needs any awareness of encoding to do that. Even worse, in the first call to `mb_strstr()`, the encoding is specified as `'UTF-16'`, which will trigger conversion to (and from) UTF-16, costing us CPU usage and memory overheads. There are a few instances where `mbstring` functions offer little-to-no added value over core functions, which we'll cover in a bit.

strpbrk()

`strpbrk()` is very similar in operation to `strstr()` and will perform a case-sensitive search that returns the remainder of the input string from the first occurrence, the difference being that `strpbrk()` treats the search string as a set: only one of the searched-for bytes needs to match. For example:

```
$ship = 'Not Invented Here';

print strpbrk($ship, "der") . PHP_EOL;
print strpbrk($ship, "red") . PHP_EOL;

Output:
ented Here
ented Here
```

Hopefully the code above should illustrate how this function works. The first string argument is searched for the first occurrence of any byte in the second string. In both cases that byte is the ASCII letter "e", it does not matter how the bytes in the second string argument are arranged, and the returned sub-string begins with the first matched byte. `strpbrk()` is binary safe but (obviously) not multi-byte safe:

```
$ship = "Not Invented\x00 Here";

print strpbrk($ship, "d\x00er") . PHP_EOL;
print strpbrk("\u{142}ukasz, \u{141}ukasz", "\u{141}") . PHP_EOL;

Output:
ented Here
łukasz, Łukasz
```